

Prepzo AI Career Platform

Frontend Explanation Guide

A Complete Technology Reference for Evaluators

Covers every technology, library, pattern, and component
with definitions, code examples, and justifications

1. Technology Stack Overview

Technology	Version	Purpose
React	19.2	UI component library
TypeScript	5.9	Type-safe JavaScript
Vite	7.2	Build tool & dev server
Tailwind CSS	4.1	Utility-first CSS
Zustand	5.0	Global state management
Axios	1.13	HTTP client for API calls
React Router	7.13	Client-side routing
Framer Motion	12.38	Animations & transitions
Radix UI	Latest	Accessible UI primitives
Lucide React	0.575	Icon library
RHF + Zod	Latest	Form handling & validation
Firebase	12.12	Phone/Google auth
Three.js + R3F	Latest	3D graphics on landing
React Hot Toast	2.6	Toast notifications

2. Why React? (Not Angular, Vue, or Svelte)

Definition: React is a JavaScript library for building user interfaces using reusable components.

Why we chose React over others:

- Component Reusability: We have 30+ UI components reused across 17 pages. React's component model makes this natural.
- Largest Ecosystem: Libraries like Framer Motion, Radix UI, Three.js all have first-class React support.
- Job Market: React is the most demanded frontend skill, aligning with our career platform.
- React 19: We use the latest version with automatic batching and concurrent rendering.

Code: main.tsx - Entry Point

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { HashRouter } from "react-router-dom";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <HashRouter>
      <App />
    </HashRouter>
  </StrictMode>
);
```

```

    </HashRouter>
  </StrictMode>
);

```

- StrictMode catches bugs early by running components twice in development.
- createRoot is React 19's modern rendering API.
- HashRouter uses URL hash (#dashboard) for routing.

3. Why TypeScript? (Not Plain JavaScript)

Definition: TypeScript adds static type checking to JavaScript, catching errors before the code runs.

Why TypeScript over JavaScript:

- Catches Bugs Early: If we pass a string where a number is expected, TypeScript shows an error immediately.
- Better IDE Support: Auto-complete and refactoring work better with types.
- Team Scalability: With 17 pages and 60+ files, types act as documentation.

Code: User Interface in authStore.ts

```

export interface User {
  id: string;
  fullName: string;
  email: string;
  role: 'student' | 'admin' | 'superadmin';
  targetRole: string;
  knownTechnologies: string[];
  skillRatings?: Record<string, number>;
  isOnboarded: boolean;
  atsScore: number;
}

```

Without TypeScript, we could accidentally write `user.atsScore` (typo) and only discover it when the app crashes.

4. Why Vite? (Not Webpack or Create React App)

Definition: Vite is a build tool that serves code during development and bundles it for production.

Why Vite over Webpack/CRA:

- 10-50x Faster startup using native ES modules.
- Hot Module Replacement: Changes appear in <50ms vs 2-5 seconds.
- CRA is Deprecated: Create React App is no longer maintained.

Code: vite.config.ts

```

export default defineConfig({

```

```

plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
  alias: { "@": path.resolve(__dirname, "src") }
},
server: {
  proxy: {
    '/api': { target: 'http://localhost:5000', changeOrigin: true }
  }
}
});

```

- viteSingleFile() bundles the entire app into one HTML file.
- proxy forwards /api calls to the Node.js backend during development.

5. Why Tailwind CSS? (Not Bootstrap or Plain CSS)

Definition: Tailwind is a utility-first CSS framework where you style using small utility classes in JSX.

Why Tailwind over Bootstrap:

- No Predefined Look: Bootstrap apps all look the same. Tailwind lets us create our unique glassmorphism theme.
- Smaller Bundle: Our CSS is ~37KB vs Bootstrap's 150KB+.
- No Class Naming Conflicts: No names like .card-wrapper-inner-container.

Code: cn() Utility - utils/cn.ts

```

import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

```

This merges Tailwind classes intelligently. If a component has bg-red-500 and a parent passes bg-blue-500, twMerge keeps only bg-blue-500.

6. Project Folder Structure

frontend/src/	
main.tsx	- Entry point
App.tsx	- Root component (routing + layout)
index.css	- Global styles (37KB design system)
api/	- API communication (16 files)
axios.ts	- Axios instance with interceptors
auth.ts, resume.ts, dynamicAI.ts, jobs.ts, ...	
store/	- State management (2 files)
authStore.ts	- User auth state
appStore.ts	- App-wide state (dark mode, resume)
hooks/	- Custom hooks (2 files)

useProctoring.ts	- Camera/mic monitoring (573 lines)
useSpeech.ts	- Voice input/output
components/	- Reusable UI (13 folders, 43+ files)
ui/	- Button, Card, Input, Avatar, Tabs...
navigation/	- Sidebar, MobileNav, MobileHeader
mentor/	- AI Mentor chat widget
assessment/	- Test-taking components
pages/	- Full pages (17 files)
Dashboard.tsx	- Main workspace (108KB)
AuthPage.tsx	- Login/Signup (65KB)
AdminPanel.tsx	- Admin dashboard (82KB)
lib/firebase.ts	- Firebase phone auth
utils/	- Helper functions

7. Routing - Why Hash Router? (Not Browser Router)

Why HashRouter over BrowserRouter:

- App is deployed as a single HTML file. BrowserRouter requires server-side config. HashRouter uses `##/dashboard` which works everywhere.
- No server configuration needed.

Code: Navigation with Loading Animation

```
const handleNavigate = (page: string) => {
  const labels = {
    dashboard: 'Syncing Workspace Node',
    jobs: 'Scanning Opportunity Grid',
    assessment: 'Evaluating Skill Vectors',
  };
  setGlobalLoading(true, labels[page]);
  setTimeout(() => {
    setCurrentPage(page);
    window.location.hash = page;
    setTimeout(() => setGlobalLoading(false), 3000);
  }, 500);
};
```

Every navigation shows a premium animated loading screen with a themed message.

8. State Management - Why Zustand? (Not Redux)

Definition: State management controls shared data that multiple components need.

Why Zustand over Redux:

- 90% less boilerplate: Redux needs actions, reducers, dispatch. Zustand needs one `create()` call.
- No Provider wrapper needed.
- Built-in persistence to `localStorage`.

Why over Context API:

- Context re-renders ALL consumers on ANY change. Zustand only re-renders components using the specific value that changed.

Code: Auth Store

```
export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      user: null,
      isAuthenticated: false,
      login: (user) => set({ user, isAuthenticated: true }),
      logout: () => {
        localStorage.removeItem('prepzo-token');
        set({ user: null, isAuthenticated: false });
      },
      loginAsync: async (data) => {
        set({ isLoading: true });
        const response = await authApi.login(data);
        set({ user: response.user, isAuthenticated: true });
        return response.user;
      },
    }),
    { name: 'prepzo-auth' } // localStorage key
  )
);
```

9. API Layer - Why Axios? (Not Fetch API)

Why Axios over native fetch():

- Interceptors: Auto-attach auth tokens and handle token refresh on 401.
- Automatic JSON parsing.
- Throws on 404/500 errors (fetch doesn't).

Code: Token Refresh Interceptor

```
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401 && !request._retry) {
      request._retry = true;
      const res = await api.post('/auth/refresh', { refreshToken });
      localStorage.setItem('prepzo-token', res.data.accessToken);
      return api(originalRequest); // Retry failed request
    }
    return Promise.reject(error);
  }
);
```

If any API call gets 401 (token expired), it automatically refreshes and retries - the user never sees an error.

10. Animations - Why Framer Motion?

Why over CSS animations:

- AnimatePresence: Animate components when removed from DOM (exit animations). CSS cannot.
- Layout animations: Auto-animate list reordering.
- Gesture support: Built-in drag, hover, tap.

Code: Global Loading Overlay

```
<AnimatePresence>
  {isGlobalLoading && (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}
      transition={{ duration: 0.3 }}
      className="fixed inset-0 z-[9999] bg-[#0a0c10]/98"
    >
      <ThinkingLoader loadingText={globalLoadingText} />
    </motion.div>
  )}
</AnimatePresence>
```

11. UI Components - Why Radix UI? (Not Material UI)

Why Radix over MUI:

- No Forced Design: MUI forces Material Design. Radix has zero styling - we apply our glassmorphism theme.
- Accessibility Built-in: Keyboard navigation, screen readers handled automatically.
- Smaller Bundle: Import only what we use.

Components used: Avatar, Tabs, Select, Switch, Label, Separator, Slot

Code: Button with CVA (Class Variance Authority)

```
const buttonVariants = cva(
  "inline-flex items-center justify-center rounded-md text-sm font-medium",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive: "bg-destructive text-white hover:bg-destructive/90",
        outline: "border border-input bg-background hover:bg-accent",
        ghost: "hover:bg-accent hover:text-accent-foreground",
      },
    },
    size: {
      default: "h-10 px-4 py-2",
      sm: "h-9 px-3",
    },
  },
)
```

```

    lg: "h-11 px-8",
  },
},
}
);

```

12. Icons - Why Lucide React?

Why over Font Awesome:

- Tree-shakeable: Only used icons included in bundle.
- Each icon is a proper React component with TypeScript props.
- Consistent 24x24 grid design.

13. Forms - Why React Hook Form + Zod?

Why RHF over Formik:

- Fewer Re-renders: Uses uncontrolled inputs by default.
- Smaller Bundle: 8KB vs 12KB.

Why Zod over Yup:

- TypeScript-first: Zod infers types from schema automatically.

Used in: AuthPage, OnboardingPage, Settings.

14. Custom Hooks

useProctoring.ts - Assessment Integrity Monitor (573 lines)

Monitors: tab switching, keyboard shortcuts (Ctrl+C/V/F12), right-click, camera coverage, background noise, fullscreen exit.

```

// Detect copy/paste attempts
const handleKeyDown = (e: KeyboardEvent) => {
  if ((e.ctrlKey || e.metaKey) && ['c', 'v', 'a', 'p'].includes(e.key)) {
    e.preventDefault();
    addViolation('keyboard_shortcut', 'Attempted Ctrl+' + e.key);
  }
};

// Camera brightness check - detect covered camera
const avgBrightness = totalBrightness / sampledPixels;
if (avgBrightness < 5 && noFaceCount >= 20) {
  addViolation('camera_covered', 'Camera appears covered');
}

```

useSpeech.ts - Voice AI Interface (109 lines)

Uses Web Speech API for voice input (SpeechRecognition) and SpeechSynthesis for AI reading questions aloud during mock interviews.

15. Firebase Integration

Firebase provides phone number authentication using OTP. Handles SMS delivery globally with built-in reCAPTCHA protection.

16. 3D Graphics - Three.js + React Three Fiber

Landing page uses interactive 3D background responding to mouse movement. R3F lets us write 3D scenes as React components with JSX.

17. Toast Notifications - React Hot Toast

Custom styled toasts matching our dark glassmorphism theme. Lightweight at 3KB with promise support.

18. Pages Overview

Page	Size	Description
Dashboard.tsx	108 KB	Main workspace + analytics
AdminPanel.tsx	82 KB	Admin user/AI management
AuthPage.tsx	65 KB	Login/Signup + OTP + OAuth
JobsPage.tsx	47 KB	Job listings + matching
NetworkPage.tsx	38 KB	Professional networking
OnboardingPage.tsx	32 KB	Profile setup wizard
LandingPage.tsx	27 KB	Public homepage + 3D
CompaniesPage.tsx	22 KB	Company profiles
ApplicationsPage.tsx	22 KB	Application tracking
NotesLibrary.tsx	15 KB	Study materials
InterviewPage.tsx	10 KB	AI mock interview

19. Authentication Flow

1. User clicks Sign Up

2. AuthPage renders signup form
3. User enters email + password
4. `authStore.registerAsync()` calls backend API
5. Backend sends OTP email via Brevo
6. User enters 6-digit OTP
7. Token stored in `localStorage`
8. Axios interceptor attaches token to all future requests
9. User redirected to `OnboardingPage`

20. Layout System

Desktop: Sidebar + Main Content area using CSS flexbox.

Mobile: Bottom navigation bar + Top header for app-like experience.

Responsive breakpoints: md: (768px) and lg: (1024px).

21. Design System (index.css - 37KB)

Contains:

- CSS Custom Properties for light/dark themes
- Glassmorphism: `backdrop-blur`, semi-transparent backgrounds
- Custom scrollbar matching dark theme
- Animation keyframes: pulse, glow, float, shimmer
- Mobile-first responsive design

22. Key Design Patterns

Component Composition: Small components combine to build complex UIs.

Button + Input + Label = Form, Form + Card = AuthPage

Container/Presentational: Pages handle logic, components handle UI.

Custom Hook Pattern: Complex logic extracted into reusable hooks (`useProctoring`, `useSpeech`).

Optimistic Updates: UI updates immediately, then syncs with backend.

Summary: Why Each Technology

Technology	Alternative	Why Chosen
React	Angular/Vue	Largest ecosystem, components
TypeScript	JavaScript	Type safety, fewer bugs
Vite	Webpack/CRA	10x faster, CRA deprecated
Tailwind CSS	Bootstrap	Custom theme, smaller bundle
Zustand	Redux	90% less code, persistence
Axios	Fetch API	Interceptors, auto JSON
Hash Router	Browser Router	Single-file deployment
Framer Motion	CSS Anim.	Exit animations, gestures
Radix UI	Material UI	Unstyled, accessible
Lucide	Font Awesome	Tree-shakeable, React
RHF + Zod	Formik + Yup	Fewer re-renders, TS-first
Firebase	Twilio	Global SMS, reCAPTCHA